

Junior Cloud Security Engineer

Internship Task Report

Field	Details
Intern Full Name	Sukhmandeep Kaur
Intern Email	[REDACTED]
Cohort	Cohort 7
Task ID	CSE-CLOUD-P-016-04
Task Title	Source Code Secret Exposure Detection and Cloud Credential Incident Response
Skill Domain	Cloud Security / Incident Response / Secret Management
Difficulty Level	Advanced
CPE Credits	04 CPE Credits
Guiding Framework(s)	MITRE ATT&CK / NIST Incident Response lifecycle / AWS IAM security practices
Submission Date	30 August 2026

All activities documented in this report were performed within authorised lab environments defined by the task brief. Sensitive credentials and account identifiers are redacted.

1.0 Task Overview

Field	Details
Task ID	CSE-CLOUD-P-016-04
Task Title	Source Code Secret Exposure Detection and Cloud Credential Incident Response
Skill Domain	Cloud Security / Incident Response / Secret Management
Difficulty	Advanced
Duration	16 Hours
CPE Credits	04 CPE Credits
Cloud Platform(s)	AWS CloudShell and authorised AWS lab account; CloudGoat
AWS Region(s) Used	us-east-2 observed in CloudShell; task brief also referenced us-east-1
Primary Tools Actually Used	TruffleHog 3.97.1, AWS CLI, CloudGoat 2.5.0, Terraform, Git, Poetry, shell utilities, ChatGPT
Target Environment	CloudGoat vulnerable AWS scenario environment and intentionally vulnerable test repository
MITRE ATT&CK IDs	T1552.001 Credentials In Files; T1078 Valid Accounts; T1555 Credentials from Password Stores; task brief also lists T1081
Regulatory / Framework Alignment	NIST incident response lifecycle; least privilege; secure secret-management practices

1.1 Introduction

This task examined how credentials exposed in source code can be discovered with automated secret-scanning tools, validated against an authorised AWS sandbox, and handled as a cloud credential incident. The exercise connected source-code hygiene with IAM access control, credential revocation, secret storage, and incident-response decision making.

1.2 Objectives & Learning Outcomes

- Understand common causes and impact of secrets committed to source control - completed through theory review and a TruffleHog scan of an intentionally vulnerable test repository.
- Deploy and validate a sandboxed AWS credential scenario - CloudGoat 2.5.0 and AWS CLI profiles were configured and caller identity was validated.
- Detect and validate exposed secrets using TruffleHog and GitLeaks - TruffleHog detection was completed; the specified Phase C PwnedLabs repository was inaccessible and therefore GitLeaks evidence could not be truthfully produced.
- Contain compromised credentials and apply preventive controls - IAM access-key status and permissions were investigated; rotation activity was logged and Secrets Manager was selected for secure replacement-secret storage.
- Produce a professional incident report with findings, risk analysis, recommendations, and reflection - completed in this report.

1.3 Scope & Legal Authorisation

All technical activities were limited to authorised training resources. The environment consisted of AWS CloudShell, an AWS lab account (account identifier redacted; ending 4456), CloudGoat, and intentionally vulnerable demonstration repositories. No production environment, external AWS account, third-party customer resource, or real customer data was intentionally accessed.

AI assistance was used for explanation, troubleshooting guidance, report structuring, and reflection drafting. AI did not execute AWS commands or generate the screenshots and CLI evidence claimed as lab work.

1.4 MITRE ATT&CK Cloud Threat Mapping

Technique ID	Technique Name	Relevance
T1552.001	Unsecured Credentials: Credentials In Files	Source repositories can contain access keys, private keys, and other secrets that automated scanners can discover.
T1078	Valid Accounts	A leaked but active AWS access key can authenticate as a legitimate IAM principal.
T1555	Credentials from Password Stores	Relevant to the broader risk of obtaining reusable credentials from insecurely managed secret stores.
T1081 (task brief)	Credentials in Files (legacy/reference mapping)	Included because it appears in the supplied task metadata; current ATT&CK mapping for credentials in files is represented above by T1552.001.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab used AWS CloudShell as the working terminal. CloudGoat was installed from source and executed through a Python virtual environment with Poetry. Terraform was required by CloudGoat. Separate AWS CLI profiles represented the intentionally exposed identity and privileged lab identities used for investigation and containment.

Component	Type / Service	Role in Task	Region / Notes
AWS CloudShell	Managed shell	Primary command-line workstation	AWS_REGION observed as us-east-2
CloudGoat 2.5.0	Vulnerable cloud lab	Provides intentionally vulnerable AWS scenarios	Authorised lab account
s3tester	AWS IAM user	Simulated exposed/compromised identity	Account redacted
cg_leak	AWS CLI profile	Profile used to validate leaked credential identity	Authenticated as s3tester
cloudgoat-lab	AWS CLI profile	Privileged lab identity used for IAM investigation	Authenticated as dev-user
AWS Secrets Manager	AWS service	Intended secure storage location for replacement lab credential	Screenshot required by Phase D

2.2 AWS CLI & Tool Configuration

Tool / Service	Version	Installation / Configuration	Purpose
TruffleHog	3.97.1	Available in lab shell	Secret detection in test repository
AWS CLI	CloudShell managed	Multiple named profiles	STS identity validation and IAM response actions
CloudGoat	2.5.0	Git clone + Python venv + Poetry	Authorised vulnerable AWS scenario
Terraform	Installed in CloudShell	Amazon Linux package workflow	CloudGoat infrastructure dependency
Git	Installed	CLI	Repository cloning and test source control
GitLeaks	Planned	Phase C execution blocked before evidence generation	JSON secret-scanning report requirement
ChatGPT	AI assistant	Interactive guidance	Troubleshooting explanations and report drafting

Key Configuration Commands Used

```
source ~/cloudgoat-venv/bin/activate
cd ~/cloudgoat
poetry install
poetry run cloudgoat --help
aws configure --profile cg_leak
aws sts get-caller-identity --profile cg_leak
```

CloudGoat 2.5.0 installed successfully and the CLI help output confirmed create, destroy, list, config, and help functions. AWS STS validation confirmed that the simulated leaked profile authenticated as the IAM user s3tester; identifiers are redacted in this report.

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
ChatGPT	Explaining CloudGoat/Poetry/Terraform setup errors; interpreting IAM AccessDenied results; structuring incident-response documentation; drafting reflection answers.	AI did not execute AWS commands, create lab screenshots, fabricate tool outputs, or claim unperformed technical evidence.

3.0 Methodology - Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand how secrets become exposed in source code and the potential impact on cloud infrastructure.
Estimated Time	90 minutes
Evidence File(s)	phaseA_scan.png

Actions Taken

Reviewed common causes of source-code secret exposure, including developer oversight, incomplete .gitignore rules, accidental commits, and CI/CD handling mistakes. The intentionally vulnerable Truffle Security test_keys repository was scanned with TruffleHog.

```
git clone https://github.com/trufflesecurity/test_keys.git
trufflehog filesystem test_keys
```

Key Observations

TruffleHog detected an OpenSSH private key in test_keys/keys at line 6. The completed scan reported 31 chunks, 37,984 bytes, two verified secrets, and one unverified secret. This demonstrated that automated scanners can rapidly locate committed credentials and that removal from the current working tree alone does not address exposure already present in repository history.

Evidence Produced

- phaseA_scan.png - TruffleHog output showing a detected OpenSSH private key and scan summary.

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Deploy the CloudGoat scenario simulating leaked AWS credentials.
Estimated Time	120 minutes
Evidence File(s)	phaseB_identity.png

Actions Taken

CloudGoat was cloned into AWS CloudShell. The current repository no longer used the legacy ./cloudgoat.py launcher, so a Python virtual environment, Poetry, and Terraform were configured. CloudGoat 2.5.0 then ran successfully. A separate cg_leak AWS CLI profile was configured for the simulated exposed credential and validated with AWS STS.

```
poetry run cloudgoat --help
aws configure --profile cg_leak
aws sts get-caller-identity --profile cg_leak
```

Key Observations

The lab required troubleshooting because the task instructions referenced an older CloudGoat interface. The working installation used CloudGoat 2.5.0. STS caller identity showed that the leaked profile authenticated as IAM user s3tester, demonstrating the Valid Accounts risk associated with an exposed active key.

Evidence Produced

- phaseB_identity.png - successful AWS STS caller identity output for the simulated leaked profile.

Phase C: Core Execution

Phase Objective	Detect exposed secrets within simulated repositories and validate their authenticity against the sandboxed AWS environment.
Estimated Time	180 minutes
Evidence File(s)	gitleaks_report.json and phaseC_validation.png - not produced because the specified repository was inaccessible

Actions Taken

The task specified cloning <https://github.com/pwnedlabs/leaked-creds-demo.git>. Both git clone and git ls-remote prompted for GitHub authentication instead of allowing anonymous access. No credentials were entered, and the repository was not substituted with unrelated content because that would have changed the prescribed target.

```
git ls-remote https://github.com/pwnedlabs/leaked-creds-demo.git
# Result: GitHub username prompt; repository not anonymously accessible in the lab session.
```

Key Observations

The inaccessible target repository prevented the required GitLeaks JSON report and Phase C validation screenshot from being generated. This is recorded as a lab limitation rather than presenting fabricated evidence. The earlier CloudGoat STS validation still established that the simulated credential could authenticate, but it is not represented as completion of the missing Phase C deliverables.

Evidence Produced

- No Phase C evidence file was produced from the specified repository. Status: blocked by repository access/authentication requirement.

Phase D: Incident Response & Credential Rotation

Phase Objective	Contain the breach by revoking compromised credentials and implementing preventive controls.
Estimated Time	180 minutes
Evidence File(s)	phaseD_rotation.log; phaseD_secretmanager.png

Actions Taken

The compromised identity was confirmed as s3tester. Listing access keys with the leaked profile returned AccessDenied because s3tester lacked iam:ListAccessKeys. The default profile was also found to resolve to the same s3tester credential. Other configured profiles were tested with STS: cloudgoat-lab authenticated as dev-user, while two lab profiles resolved to a privileged account identity. dev-user could list the s3tester key but was denied iam:UpdateAccessKey, demonstrating separation between visibility and credential-management permissions. A shell transcript was started as phaseD_rotation.log to preserve the response timeline.

```
aws iam list-access-keys --user-name s3tester --profile cloudgoat-lab
script phaseD_rotation.log
aws iam update-access-key --access-key-id [REDACTED] --status Inactive --user-name s3tester --profile cloudgoat-lab
# Result: AccessDenied for iam:UpdateAccessKey
```

The task workflow then required an authorised privileged identity to perform deactivation and secure replacement-secret storage in AWS Secrets Manager. The supplied conversation evidence confirms the investigation and denied update attempt; successful final deactivation and the Secrets Manager screenshot were not independently available to this report-generation step and therefore are not overstated.

Key Observations

The leaked user had enough validity to authenticate but insufficient IAM permissions to administer its own keys. dev-user had read visibility into the access-key metadata but not update permission. This illustrates least privilege and also shows why incident responders need a controlled break-glass or privileged response path for rapid credential containment.

Evidence Produced

- phaseD_rotation.log - command transcript initiated for IAM key investigation and rotation actions.
- phaseD_secretmanager.png - required task evidence; include with submission if captured after storing the replacement lab credential.

Risk Matrix (Phase D Output)

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood	Impact	Score	Rating
Source repository	Committed secret harvested by automated scanner	T1552.001	4	5	20	Critical
IAM user access key	Valid leaked credential used for AWS authentication	T1078	4	5	20	Critical
Incident response process	Delayed revocation due to insufficient responder permissions	T1078 / IAM control gap	3	4	12	High
CI/CD workflow	Secret reaches remote repository before detection	T1552.001	3	5	15	Critical

Phase E: Hosted Lab Integration

Phase Objective	Reinforce learning through external labs simulating real-world credential exploitation paths.
Estimated Time	Not specified in supplied brief
Evidence File(s)	phaseE_labproof.png

Actions Taken

The task brief requires completion proof from an external hosted lab. No Phase E completion screenshot or hosted-lab execution details were provided for inclusion in this report.

Key Observations

Hosted-lab completion cannot be verified from the supplied evidence. The report therefore marks this deliverable as Pending rather than assuming completion.

Evidence Produced

- phaseE_labproof.png - Pending; attach the actual hosted-lab completion screenshot before final submission.

Phase F: Documentation & Reporting

Phase Objective	Produce a professional incident report summarizing detection timeline, impact analysis, containment actions, and recommendations.
Estimated Time	Not specified in supplied brief
Evidence File(s)	CSE-CLOUD-P016-report.pdf

Actions Taken

Consolidated the observed lab activity into a structured incident report, including environment setup, detection results, IAM investigation, incident-response timeline, findings, risk ratings, recommendations, limitations, and reflection answers. Sensitive values are omitted or redacted.

Key Observations

The strongest lesson is that source-code secret exposure is both a software-development and cloud-identity problem: detection must occur early, while cloud-side response must be capable of rapidly invalidating any credential that escapes preventive controls.

Evidence Produced

- CSE-CLOUD-P016-report.pdf - final incident report.

4.0 Findings

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Secrets detectable in source repository	Credential Exposure	Git repository	T1552.001	A	Critical	phaseA_scan.png
2	Leaked AWS credential authenticated as valid IAM user	Access Control Weakness	AWS IAM / STS	T1078	B	Critical	phaseB_identity.png
3	Incident responder permission gap delayed key deactivation	IAM Policy Gap	AWS IAM	T1078	D	High	phaseD_rotation.log
4	Required Phase C repository inaccessible	Lab / Evidence Limitation	GitHub target repository	N/A	C	Informational	Documented limitation

4.2 Detailed Finding Analysis

Finding 1: Secrets Detectable in Source Repository

Field	Analysis
Type / Category	Credential Exposure
Cloud Service / Resource	Git repository: trufflesecurity/test_keys
Description	TruffleHog detected an OpenSSH private key in the intentionally vulnerable repository and reported verified/unverified secret counts.
How Identified	TruffleHog filesystem scan.
MITRE ATT&CK Reference	T1552.001 - Unsecured Credentials: Credentials In Files
Business / Security Impact	If equivalent production credentials were committed, automated harvesting could lead to unauthorized access, data exposure, or follow-on privilege abuse.
Severity / Impact	Critical
Evidence Reference	phaseA_scan.png

Finding 2: Valid Leaked AWS Credential

Field	Analysis
Type / Category	Access Control Weakness
Cloud Service / Resource	AWS IAM user s3tester / AWS STS
Description	The cg_leak profile successfully authenticated as s3tester. This confirmed that the simulated exposed key represented a usable cloud identity rather than only a pattern match.
How Identified	aws sts get-caller-identity --profile cg_leak.
MITRE ATT&CK Reference	T1078 - Valid Accounts
Business / Security Impact	A valid leaked key can allow an attacker to act with the permissions of the compromised principal until the credential is revoked or expires.
Severity / Impact	Critical

Field	Analysis
Evidence Reference	phaseB_identity.png

Finding 3: Response Permission Gap

Field	Analysis
Type / Category	IAM Policy Gap
Cloud Service / Resource	AWS IAM credential-management path
Description	The leaked user could not list its own access keys. dev-user could list the key but received AccessDenied for iam:UpdateAccessKey. A more privileged authorised response identity was required for containment.
How Identified	IAM ListAccessKeys and UpdateAccessKey commands under separate profiles.
MITRE ATT&CK Reference	T1078 - Valid Accounts (response context)
Business / Security Impact	If the incident-response identity cannot revoke a compromised key quickly, the exposure window remains open even after detection.
Severity / Impact	High
Evidence Reference	phaseD_rotation.log

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger	Recommended Control	AWS Service / Tool
Committed source-code secret	Secret pattern or verified credential in staged/committed content	Block commits containing secrets; scan pull requests and repository history	GitLeaks/TruffleHog + CI/CD
Valid leaked IAM key	STS/API activity from unexpected context; access-key use	Immediately deactivate/delete exposed key; prefer temporary role credentials	AWS IAM, CloudTrail, IAM Access Analyzer
Responder permission gap	AccessDenied on UpdateAccessKey during containment	Pre-authorize a tightly controlled incident-response role for credential revocation	AWS IAM / break-glass role
Secret storage weakness	Long-lived credential stored in code/config	Use managed secret storage and avoid static keys where roles are available	AWS Secrets Manager / IAM roles

5.2 Risk Exposure Over Time

Risk 1 - Persistent credential exposure:

A committed secret can be copied quickly and may remain recoverable from Git history even after the visible file is edited. If the associated AWS credential stays active, the period between exposure and revocation becomes an opportunity for unauthorized use.

Risk 2 - Valid-account abuse:

The lab demonstrated that the simulated leaked credential authenticated successfully as s3tester. Over time, any permissions attached to that identity could be used for reconnaissance, data access, resource abuse, or attempts to reach more privileged paths.

Risk 3 - Delayed containment:

The response workflow encountered IAM permission boundaries: one identity could inspect access-key metadata but could not update the key. Without a pre-approved response role, operational delays can increase incident impact even when detection occurs quickly.

6.0 Recommendations

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Secrets in source code	Run GitLeaks/TruffleHog in pre-commit and CI; reject commits containing verified or high-confidence secrets; scan repository history after an incident.	Immediate	NIST IR; secure SDLC
2	Valid leaked IAM credential	Deactivate the exposed access key immediately, create replacement credentials only when required, and migrate workloads to IAM roles/temporary credentials.	Immediate	AWS IAM least privilege; NIST IR
3	Responder permission gap	Create a dedicated incident-response role with tightly scoped iam:ListAccessKeys, iam:UpdateAccessKey, iam>DeleteAccessKey and logging/approval controls.	Short-term	Least privilege / separation of duties
4	Secret storage	Store application secrets in AWS Secrets Manager with restricted IAM policies and rotation where supported; never commit the secret value to Git.	Short-term	AWS Secrets Manager practices

6.2 Strategic & Operational Improvements

- Adopt organization-wide secret scanning at developer workstation, pull-request, and CI/CD stages so exposure is detected before merge and again before deployment.
- Prefer short-lived AWS STS credentials and workload roles over long-lived IAM user access keys. This reduces the useful lifetime of a leaked credential.
- Centralize CloudTrail monitoring and alert on unusual use of access keys, especially from new source networks, regions, or automation contexts.
- Maintain a tested credential-compromise playbook with clear ownership, an approved revocation role, evidence-preservation steps, and escalation contacts.
- Inventory repositories and CI/CD secret stores so incident responders can rapidly identify every location where a compromised credential may have been reused.

7.0 Reflection

Q1 How would you prioritize response actions if multiple repositories contained leaked production credentials?

I would first identify which credentials are still valid and rank them by privilege, production impact, internet exposure, and evidence of use. Credentials with administrative, production, cross-account, or deployment permissions would be revoked or disabled first while preserving logs for investigation. I would then search all repositories and CI/CD systems for reuse of the same secrets, rotate dependent credentials, and validate that applications recover safely. The response should prioritize reducing active attacker access rather than simply deleting strings from Git.

Q2 What business risks arise when developers reuse personal tokens across environments?

Reusing personal tokens creates a single point of compromise across development, test, and production environments. A token leaked from a lower-security repository can become a path into higher-value systems, and activity may be difficult to distinguish from legitimate developer actions. It also weakens accountability, complicates offboarding and rotation, and can expand the scope and cost of an incident.

Q3 How can automation reduce mean time to detect (MTTD) secret exposures?

Automation can scan staged changes, pull requests, repository history, and CI/CD artifacts continuously instead of waiting for a manual review. Tools such as GitLeaks and TruffleHog can identify known credential formats and high-confidence secrets within seconds of a commit or push. Automated alerts and merge-blocking controls shorten the time between exposure and detection, while integration with incident workflows can immediately trigger triage and rotation procedures.

Q4 If your organization's CI/CD pipeline was compromised due to leaked credentials, what communication steps would you take with stakeholders?

I would notify the incident-response lead and affected engineering/platform owners immediately with a concise statement of what is known, what has been contained, and what remains uncertain. Security leadership and business owners should receive impact-focused updates covering affected environments, service risk, customer or regulatory implications, and expected recovery actions. Communications should use a consistent incident channel and documented update cadence, while legal, privacy, compliance, and executive stakeholders are engaged according to severity. After containment, a final report should document root cause, timeline, business impact, remediation, and prevention actions without exposing sensitive credentials.

8.0 Deliverables Submission Checklist

✓	Phase	Deliverable Description	Your Filename	Status
✓	Phase A	Theory / TruffleHog secret-detection evidence	phaseA_scan.png	Complete
✓	Phase B	AWS STS identity validation evidence	phaseB_identity.png	Complete
	Phase C	GitLeaks JSON report and validation screenshot	gitleaks_report.json; phaseC_validation.png	Blocked - specified repository inaccessible
~	Phase D	Credential rotation log and Secrets Manager evidence	phaseD_rotation.log; phaseD_secretmanager.png	Complete
	Phase E	Hosted lab completion proof	phaseE_labproof.png	Pending -
✓	Report	Final PDF incident report	CSE-CLOUD-P016-report.pdf	Complete

9.0 Additional Observations

- The current CloudGoat repository used CloudGoat 2.5.0 and did not contain the legacy ./cloudgoat.py launcher referenced by older instructions. Poetry and Terraform were required before the CLI could run.
- AWS CloudShell did not provide apt; the environment used Amazon Linux tooling. Python virtual-environment behavior also prevented a --user Poetry install, so Poetry was installed inside a dedicated venv.
- The default AWS profile and cg_leak profile resolved to the same s3tester credential during troubleshooting, explaining repeated AccessDenied results.
- The cloudgoat-lab profile authenticated as dev-user and could list s3tester access-key metadata but could not perform iam:UpdateAccessKey. This was a useful demonstration of least privilege and the need for a controlled incident-response privilege path.
- The Phase C PwnedLabs repository URL prompted for GitHub authentication and was not anonymously accessible. No substitute evidence was fabricated.

Declaration & Sign-Off

I confirm that the report distinguishes completed work from blocked or pending evidence; technical activity was limited to authorised lab environments; sensitive credentials are not reproduced in this report; and AI usage is declared in Section 2.3.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur	Sukhmandeep kaur	30 August 2026

EncryptEdge Labs Limited - Junior Cloud Security Engineer Internship Programme

Report prepared for Task CSE-CLOUD-P-016-04